UNIVERSIDADE DE FORTALEZA
VICE REITORIA DE GRADUAÇÃO
CENTRO DE CIÊNCIAS TECNOLÓGICAS
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

T300 - Programação Funcional

Prof Bruno Lopes
bruno.batista@unifor.br

Este instrumento de avaliação corresponde a um projeto prático individual de Programação Funcional com Elixir. O aluno deverá implementar uma aplicação de Linha de comando (CLI) para gerenciamento de uma agenda de contatos pessoais.

Deverá ser submetido um arquivo ZIP contendo o **código fonte do projeto e o arquivo PDF que conste todos os prompts utilizados com a IA e suas respectivas respostas**. Alguns cuidados que devem ser observados ao realizar a atividade:

- Certifique-se de que a aplicação esteja funcionando corretamente. **Entregas com aplicação que não inicializem terão desconto de 5,0 pontos.**
- A aplicação deverá ser apresentada presencialmente ao professor no dia especificado, onde o professor irá realizar uma arguição sobre o projeto. **Caso não seja apresentado o projeto, automaticamente será dada a nota zero**.
- Não serão aceitos outros meios de submissão. **Envios fora do prazo terão desconto de 2,0 pontos por dia de atraso.**
- A atividade deve ser realizada de forma individual. O uso de IA é permitido desde que seja especificado no PDF e que conste todos os prompts e as respostas das respostas da IA. **Plágio, cópia indiscriminada da IA sem apontar seu uso resultará em nota zero**.

Regra de Ouro (Apresentação e Defesa Oral)

ATENÇÃO: O envio do código não garante aprovação. No momento da apresentação individual com o professor, cada estudante será arguido sobre o código que produziu, suas decisões de arquitetura e o funcionamento dos recursos Elixir utilizados. A não demonstração de domínio técnico ou incapacidade de explicar o próprio código resultará na **INVALIDAÇÃO IMEDIATA** de todo o trabalho e atribuição de **NOTA ZERO (0,0)**, independentemente de o projeto estar em pleno funcionamento.

Quaisquer dúvidas ou algum detalhe que não foi abordado explicitamente ou implicitamente neste documento devem ser dirimidos com o professor da componente curricular antes do término do prazo de submissão do trabalho via **AVA**.

Diretrizes para Uso Ético de Inteligência Artificial (Obrigatório)

O uso de ferramentas de Inteligência Artificial Generativa (IAG) nesta atividade é regulamentado e deve seguir rigorosamente as normas institucionais:

- **Respeito ao Guia Institucional:** É obrigatório a leitura e o cumprimento integral do **"Guia de Uso Ético de Inteligência Artificial no Ensino Superior"** disponibilizado pelo professor no AVA da turma.
- **Níveis Autorizados:** A atividade admite uso enquadrado entre os Níveis 1 a 3 (Consulta conceitual, Brainstorming/planejamento ou Assistência com revisão). O Nível 4 só é aceito mediante apresentação do log completo e detalhado do processo de cocriação.
- **Limites Inegociáveis:** A IA NÃO deve escrever a solução completa da atividade nem gerar código que você não compreenda linha a linha. É expressamente proibido inserir dados sensíveis ou credenciais em ferramentas de IA.
- **Entrega Obrigatória da Declaração:** Juntamente com a entrega do repositório, cada estudante deve preencher, assinar e anexar no AVA o documento **"[MODELO] - Declaração de Uso de Inteligência Artificial"** (também disponível no AVA).
- **Preenchimento Detalhado (Seção 4 da Declaração):** A seção 4 da declaração exige descrição clara e objetiva de quais perguntas foram feitas, respostas obtidas e como foram aplicadas no código. Respostas genéricas como "usei para tirar dúvidas" ou declarações incompletas resultarão na **atribuição de nota zero (0,0)** à atividade, sem direito a reenvio.
- **Assinatura:** A declaração pode ser assinada digitalmente com a conta gov.br (nível Prata ou Ouro via assinador.iti.br) ou de forma manuscrita escaneada.

“Não tente.
Faça ou não faça.
Não existe tentativa”



VISÃO GERAL DO PROJETO

Nesta atividade, você irá desenvolver uma aplicação de linha de comando (CLI) para o gerenciamento de uma agenda de contatos pessoais. A aplicação deve ser construída inteiramente em Elixir, utilizando o gerenciador de projetos Mix e os princípios de paradigma funcional estudados em aula.
A atividade tem como objetivo consolidar os seguintes conceitos:

- Imutabilidade e transformação de dados com Enum e pipe operator (|>)
- Pattern matching em cláusulas de função para parsing de comandos e flags
- Recursão de cauda como mecanismo de loop interativo
- Persistência de estado entre sessões via serialização JSON
- Organização de código em módulos com responsabilidades bem definidas

ESPECIFICAÇÕES DO SISTEMA

Modelo de Dados

Cada contato deve armazenar os seguintes campos:

Campo	Tipo	Observação
id	integer	Gerado automaticamente (timestamp em ms)
name	string	Nome completo da pessoa
company	string	Nome da empresa ou local de trabalho
phone	string	DDD + número (ex: 85912345678)
email	string	Endereço de e-mail válido

Comandos da Aplicação

A aplicação deve implementar os seguintes comandos no loop interativo:

Comando	Exemplo	Descrição
add	`add --name Ana Lima --company Acme --phone 85912345678 --email ana.lima@acme.com`	Adiciona um novo contato
edit <id>	`edit 123 --phone 85912341234`	Edita campos(s) de um contato existente
del <id>	`del 123`	Remove um contato pelo sei ID
show <id>	`show 123`	Exibe todos os dados de um contato

list	`list`	Lista todos os contatos (id + nome + empresa + telefone + email)
search	`search --name Ana`	Busca contatos por nome, telefone ou e-mail
exit	`exit`	Encerra a aplicação

Detalhamento do Comando search

O comando search aceita uma flag que indica o campo de busca. A busca deve ser case-insensitive e por substring (busca parcial). Apenas uma flag deve ser aceita por vez. Por exemplo:

```
# Busca por nome (parcial, case-insensitive)
agenda> search --name ana

# Busca por número de telefone (parcial, e.g.: busca por DDD)
agenda> search --phone 85

# Busca por endereço de e-mail (parcial)
agenda> search --email gmail
```

Para o parsing das flags, recomenda-se implementar uma função *`parse_search/1`* que receba o restante das string após "search e retorne uma tupla *{:name, valor}*, *{:phone, valor}* ou *{:email, valor}*.

Detalhamento do Comando edit

O comando edit deve permitir alterar um ou mais campos de um contato já existente, sem necessidade de redigitar todos os dados. As mesmas flags de search são usadas para indicar qual campo alterar. Por exemplo:

```
# Alterar apenas o telefone
agenda> edit 123 --phone 85912345678

# Alterar apenas o e-mail
agenda> edit 123 --email novo.email@acme.com

# Alterar nome e empresa
agenda> edit 123 --name Ana Silva --company Acme LTDA
```

Persistência de Dados

Os dados devem ser salvos em um arquivo **contacts.json** no diretório de execução da aplicação. O arquivo deve ser lido ao iniciar a aplicação e atualizado a cada operação de escrita (add, edit, del).

Exemplo de estrutura do arquivo JSON:

```
[
	{
		"id": 1713531600000,
		"name": "Ana Lima",
		"company": "Acme Ltda",
		"phone": "85912345678",
		"email": "ana.lima@acme.com"
	},
	{
		"id": 1713531601000,
		"name": "José Silva",
		"company": "Acme Ltda",
		"phone": "85912348899",
		"email": "jose.silva@acme.com"
	}
]
```

ARQUITETURA ESPERADA

O projeto deve ser organizado com ao menos três módulos com responsabilidades bem separadas:

Módulo	Responsabilidade
AgendaCli	Ponto de entrada (main/1), loop interativo e parse de comandos
AgendaCli.Contacts	Funções puras de manipulação da lista (add, delete, search, edit)
AgendaCli.Store	Leitura e escrita do arquivo JSON (load/0 e save/1)

A criação de módulos adicionais é permitida e incentivada, desde que a separação de responsabilidades seja mantida.

REQUISITOS TÉCNICOS OBRIGATÓRIOS

Para que o projeto seja aceito para avaliação, os seguintes requisitos técnicos devem ser atendidos:

1. O projeto deve ser criado com ***mix new*** e executável via ***mix run***.
2. Deve utilizar a dependência ***Jason*** para serialização JSON.

3. O loop principal deve ser implementado via recursão de cauda (sem bibliotecas externas de loop).
4. O parse de comandos e flags deve usar ***pattern matching*** em cláusulas de função.
5. Nenhuma variável global ou estado mutável pode ser utilizado.
6. O código deve estar organizado nos módulos especificados neste documento.
7. O projeto deve conter um arquivo README.md com instruções de instalação e uso.

CRITÉRIOS DE AVALIAÇÃO

Critério	Pontuação	Descrição
Funcionalidades CRUD	3,0 pts	add, del, show, list funcionando corretamente
Comando search com flags	2,0 pts	Busca parcial e case-insensitive nos 3 campos
Qualidade do código	1,5 pts	Uso correto de pattern matching, pipe, imutabilidade
Persistência JSON	1,0 pts	Leitura e escrita corretas do contacts.json
README e organização	1,0 pts	Documentação, estrutura de módulos, mix.exs correto.
TOTAL	**10,0 pts**	

A arguição oral faz parte integrante da avaliação. Mais do que o código funcionar, importa que o aluno compreenda cada decisão que tomou. Durante a apresentação, o professor realizará arguição sobre qualquer trecho do código entregue. **A incapacidade de explicar decisões de implementação, respostas superficiais ou incorretas sobre o próprio código serão interpretadas como indício de autoria inidônea** — seja por cópia de terceiros ou uso acrítico de ferramentas de IA generativa — **e implicará em nota zero na atividade**."

⚠️ Avisos Importantes e Instruções de Submissão

Para fins de padronização, organização e cumprimento dos prazos regimentais, todos os estudantes devem seguir estritamente as diretrizes de envio descritas a seguir:

Formato e Conteúdo do Envio:

- A entrega deve ser realizada exclusivamente via **Ambiente Virtual de Aprendizagem (AVA)**, em formato de arquivo compactado **.zip**.
- O arquivo compactado deve conter obrigatoriamente:
    - O **código-fonte completo** do projeto React;

    - O documento PDF da **Declaração de Uso de Inteligência Artificial** devidamente preenchido e assinado (via *gov.br* ou manuscrito escaneado).

Penalidades por Não Conformidade:

- **Ausência de Assinatura na Declaração:** A entrega da Declaração de Uso de IA sem a devida assinatura formal implicará o desconto de **5,0 (cinco) pontos** na nota final do trabalho.
- **Preenchimento Vago ou Inadequado da Seção 4:** Conforme estabelecido nas normas institucionais, a ausência de detalhamento na descrição do uso de IA (respostas genéricas como *"usei para tirar dúvidas"* ou *"pedi ajuda ao ChatGPT"*) acarretará a **atribuição de nota zero (0,0)** à atividade, sem possibilidade de reenvio.
- **Uso Antiético ou Não Autorizado de IA:** O descumprimento do Guia Institucional — tal como solicitar que a ferramenta gere a solução completa, extrapolar os níveis autorizados (1 a 3) sem o log completo de cocriação, ou a incapacidade de explicar o código linha a linha na defesa oral — resultará na **invalidação total do trabalho com nota zero (0,0)**, além de eventuais encaminhamentos disciplinares.

Canal de Submissão e Política de Atrasos:

- **Canal Exclusivo:** O AVA é o único meio aceito para envio da atividade. Trabalhos encaminhados por e-mail, redes sociais, mensagens instantâneas ou qualquer outro canal **não serão recebidos nem avaliados**.
- **Submissões Fora do Prazo:** Envios posteriores à data-limite cadastrada no AVA serão aceitos, contudo, incidirá um desconto progressivo de **2,0 (dois) pontos na nota final para cada dia de atraso**.

🚧 LEMBRE-SE

A elegância da programação funcional está na composição de funções pequenas e puras. Prefira muitas funções simples a poucas funções complexas. Use o pipe operator (|>) para encadear transformações e torne o seu código legível como uma receita de passos.